# Turtlzim

Suivi d'avancement du projet

état au 16 octobre 2026 — dernier commit 3c89149+

## À quoi sert ce document

Turtlzim est une application de notes dont les notes sont au format wiki de Zim, avec chiffrement par carnet, en trois versions (web, bureau, mobile) qui partagent un seul code. Ce document garde la trace de **ce qui a été fait, ce qui est en cours, en attente, reporté ou écarté, et ce qui reste à faire pour chaque point**. Il est repris et recompilé à chaque étape, pour qu'on puisse relire d'un coup d'œil où l'on en est. Les chiffres de tests et les mots « vérifié » ou « non vérifié » ne disent que ce qui a réellement été exécuté ; ce qui n'a jamais été essayé est dit comme tel (section 5).

### Légende

| | |
|---|---|
| FAIT | Fait et vérifié par des tests (le « reste à faire » dit ce qui manque encore autour). |
| EN COURS | Commencé, pas terminé. |
| EN ATTENTE | Prévu, pas commencé : il manque du temps, un outil, une réponse de votre part ou une autre étape. |
| REPORTÉ | Laissé pour plus tard, volontairement : ça a du sens, mais pas maintenant. |
| ÉCARTÉ | Abandonné ou remplacé, avec la raison : on n'y reviendra pas sans nouvelle décision. |

### Synthèse

| | | | | | |
|---|---|---|---|---|---|
| FAIT | 32 | EN COURS | 0 | EN ATTENTE | 7 |
| REPORTÉ | 3 | ÉCARTÉ | 8 | | |

**Dernier relevé des tests** (au 16 octobre 2026) : `cargo test` du format, des règles du carnet et de leurs liaisons wasm : 297 réussis (9 ignorés, qui demandent le corpus de Zim) ; tests unitaires de l'application web : 580 réussis (5 ignorés) ; tests du serveur de comptes : 108 réussis ; tests du cœur Turtl (chiffrement, coffre, synchro) : 121 en natif, 79 en wasm32 (tous réussis) ; scénarios dans un vrai navigateur : 133 sur 133(Chromium) et 117 sur 117 (un scénario corrigé, relancé seul)(WebKit, le moteur d'un iPhone).

## 1 Où on en est, point par point

| N° | Point | État | Où on en est | Reste à faire |
|---|---|---|---|---|
| **A — Fondations (format, cœur, port WebAssembly)** | | | | |
| A1 | **Format Zim : lecture et écriture** | FAIT | Un lecteur/écrivain du format Zim (`core/zim-format`), réécrit sans copier le code de Zim. Un fichier relu puis réécrit est identique octet pour octet : test d'aléa à chaque `cargo test`, et test sur de vrais carnets de Zim (`scripts/zim-golden.sh`). Deux défauts trouvés par ces tests ont été corrigés. | Aucune relecture extérieure. Le corpus d'essai est celui que Zim fournit : d'autres carnets réels pourraient montrer des cas rares. |
| A2 | **Cœur Turtl en Rust remis en état** | FAIT | Le cœur de Turtl (dormant, outils anciens) compile et se teste avec les outils actuels (édition 2018, async partout). Documenté dans `docs/core-rs-revival-notes.md`. | Mettre à jour la libsodium d'origine du build natif (la 1.0.16 est notée dans `CLAUDE.md` ; à vérifier où elle sert encore). |
| A3 | **Port WebAssembly du cœur** | FAIT | Chiffrement par la vraie libsodium (libsodium.js, mêmes octets que le natif, vérifié), SQLite réel compilé en wasm, réseau asynchrone, stockage persistant (OPFS), le tout dans un Worker. | Rien de connu côté web. Le pendant natif pour les téléphones est le point D5. |

| N° | Point | État | Où on en est | Reste à faire |
|---|---|---|---|---|
| A4 | **Règles du carnet** | FAIT | Noms, liens, arbre, rétroliens, recherche, renommage qui garde les liens, étiquettes, journal, complétion des liens, tableaux : un seul endroit (`core/zim-notebook`, `zim-format`), appelé par l'application via wasm. | Comparer plus largement avec le comportement de Zim sur des carnets réels (aujourd'hui : corpus de Zim seulement). |
| **B — Application web** | | | | |
| B1 | **Éditeur (CodeMirror 6)** | FAIT | Mise en forme vivante du texte Zim (le document *est* le fichier, jamais un arbre réécrit), raccourcis, tableaux en grille, images, en-tête repliée. | Fidélité complète à l'éditeur de Zim (risque 7 de `CLAUDE.md`) : syntaxes rares et réglages de Zim non reproduits. |
| B2 | **Carnet : pages, liens, recherche** | FAIT | Arbre des pages, rétroliens, recherche, renommer/supprimer, import de dossier ou de zip, export en zip. | — |
| B3 | **Pièces jointes et images** | FAIT | Joindre, glisser-déposer, coller ; images dessinées dans la page ; une revue a trouvé et fait corriger un SVG qui s'exécutait. | — |
| B4 | **Journal, étiquettes, sommaire, tableaux** | FAIT | Calendrier du journal, panneau d'étiquettes, sommaire de la page, complétion de `[[liens]]`, tableaux. | — |
| B5 | **Écriture sur téléphone** | FAIT | Barre de mise en forme, appuis tactiles, mise en page qui suit le clavier ; testé avec des événements tactiles et un clavier simulé, en Chromium et en WebKit. | **Jamais essayé sur un vrai téléphone** (seulement l'émulateur Android) : le comportement du clavier réel peut différer. |
| B6 | **Application installable, hors ligne, code épinglé** | FAIT | Service worker qui liste chaque fichier avec son empreinte et ne lance que la version approuvée ; mises à jour qui attendent l'accord. | Le worker lui-même vient de l'hébergeur : un hôte hostile n'est pas arrêté, seulement rendu vérifiable (risque 6). Le paquet de bureau est la garantie la plus forte. |
| **C — Chiffrement local** | | | | |
| C1 | **Carnets chiffrés (coffre)** | FAIT | Cœur Turtl en wasm dans un Worker, une phrase secrète (scrypt, 128 Mio), un compte local, un « espace » par carnet ; rien de lisible ailleurs que dans le coffre (un test scanne tout ce que garde le navigateur). | Mesurer le déverrouillage sur de vrais téléphones (181 s sur l'émulateur Android à deux cœurs émulés). |
| C2 | **Changer la phrase, sauvegarder, restaurer** | FAIT | Changement de phrase repris après un arrêt en cours de route ; sauvegarde du coffre en un seul fichier toujours chiffré ; restauration dans un autre navigateur ; coupures testées à plusieurs moments. | Une sauvegarde ne suit pas un changement de phrase (documenté) ; clés des carnets non renouvelées par un changement de phrase. |
| C3 | **Un seul chemin de chiffrement** | ÉCARTÉ | Règle tenue partout : libsodium seule, jamais un second code de chiffrement par plateforme. | Écarté : tout chiffrement refait à la main ou par plateforme. |
| **D — Applications (bureau, mobile)** | | | | |
| D1 | **Bureau Windows et Linux (Tauri 2)** | FAIT | La même application web dans Tauri ; paquets `.deb`, `.rpm`, `.AppImage`, NSIS et MSI construits par GitHub Actions à chaque changement. | macOS non fait ; signature des installeurs ; mise à jour automatique ; **jamais lancé sur une vraie machine Windows** (la CI seulement). |
| D2 | **Carnets en vrais dossiers (bureau)** | FAIT | Un carnet est un dossier Zim lu et écrit sur place, changement tout ou rien avec journal de reprise ; Zim peut ouvrir le même dossier. | Modification simultanée avec Zim lui-même : non essayée. |
| D3 | **Android (Capacitor)** | FAIT | APK de débogage, essayé dans une vraie WebView sur émulateur (douze vérifications automatiques) ; aucune permission réseau, pas de sauvegarde automatique. | Signature de publication, boutique, `appId` provisoire, essai sur téléphone réel ; la permission `INTERNET` sera nécessaire pour les comptes (point E17). |
| D4 | **iPhone / iPad** | EN ATTENTE | Rien de commencé : le même principe (Capacitor) s'applique, WebKit est déjà testé. | Il faut un Mac avec Xcode (ou un macOS en CI), puis un essai sur appareil. |

| N° | Point | État | Où on en est | Reste à faire |
|---|---|---|---|---|
| D5 | **Cœur natif relié par JNI / FFI** | REPORTÉ | Les coques font tourner le cœur wasm dans leur WebView : ça marche et évite deux chemins de code. | À reprendre si le coffre est trop lent sur téléphone (point C1) ; le dépôt de référence `turtl/mobile` montre la liaison Android. |
| D6 | **Application Flutter / Dart** | ÉCARTÉ | Question posée le 13 octobre : non, l'éditeur serait à réécrire (plusieurs semaines, deux éditeurs à garder identiques). | Écarté. À rouvrir seulement si l'éditeur dans une WebView s'avérait inacceptable sur appareil réel. |
| D7 | **Interfaces natives GTK / Qt** | ÉCARTÉ | Décision du 5 octobre : une interface web partagée plutôt que trois interfaces. | Écarté. |
| **E — Comptes, serveur et sauvegardes en ligne** | | | | |
| E1 | **Étude de faisabilité** | FAIT | Étude en français (`docs/zim-cloud-feasibility.md`) : hébergement, comptes, connecteurs, chiffrement, feuille de route en lots 0 à 8, 9 décisions ouvertes, risques. | Les 9 décisions ouvertes (point E18). |
| E2 | **Hébergement mutualisé OVH** | ÉCARTÉ | Lot 0 de l'étude (essai d'hébergement) abandonné le 15 octobre : on continue sur ce serveur. | Écarté pour l'instant ; un VPS sera acheté « si le projet aboutit » (point E3). |
| E3 | **Serveur de comptes sur cette machine** | FAIT | `server/` : Node 24, SQLite intégré, sert l'application et `/api/`. Tourne sur le port 8443 (https avec certificat local). | Mise en service pour d'autres personnes : VPS, nom de domaine, https de confiance, sauvegarde de la base, supervision, mises à jour. Rien de tout cela n'est fait. |
| E4 | **Serveur de synchro d'origine de Turtl** | ÉCARTÉ | Remplacé de fait par `server/` (Node 24 + SQLite, protocole à nous) ; l'ancien (Node 8 + Postgres) n'a pas été revu. | Écarté ; `CLAUDE.md` le cite encore parmi les « décisions verrouillées » : à corriger avec votre accord. |
| E5 | **Comptes par e-mail, code à l'inscription** | FAIT | Inscription (adresse + mot de passe + code à 6 chiffres envoyé une seule fois), connexion, mot de passe oublié, export des données, suppression ; mots de passe en scrypt, sessions en cookie `HttpOnly`, limites de tentatives, protections contre le vol de session et l'énumération. | **Envoi réel d'e-mails jamais essayé** (sans serveur SMTP, les codes vont dans un fichier) ; SPF/DKIM ; changer le mot de passe depuis l'interface, « se déconnecter partout », vue d'administration. |
| E6 | **Code à chaque connexion** | ÉCARTÉ | Votre décision du 15 octobre : le code seulement à l'inscription. | Écarté. |
| E7 | **Connexion Google et GitHub** | FAIT | Flux OAuth complet côté serveur (secret jamais dans la page, PKCE, état à usage unique, liaison au navigateur, adresses confirmées par le service seulement). | **Jamais essayé avec les vrais services** : à vous de créer les deux applications OAuth (étapes dans `docs/zim-server.md`), puis un essai réel. |
| E8 | **Facebook, LinkedIn, Microsoft, GitLab (connexion)** | REPORTÉ | Décision du 15 octobre : après Google et GitHub. | Un pilote par service, même schéma qu'aujourd'hui ; une application OAuth à créer chez chacun. |
| E9 | **Synchronisation en clair vers le compte (1 Mo)** | FAIT | « Garder ce carnet sur le serveur » : envoi automatique, récupération, refus d'une écriture faite depuis une ancienne révision ; depuis le 16 octobre la synchronisation se fait *page par page* (point E16a) : seules les pages changées voyagent, des pages différentes changées sur deux appareils se fusionnent seules. Essai à deux navigateurs avec le vrai serveur. | Fusion à l'intérieur d'une page (aujourd'hui une même page changée des deux côtés est un conflit, avec « garder les deux ») ; quota par carnet ; les carnets de dossier du bureau et le téléphone ne se synchronisent pas encore ; la liste des empreintes tient dans le `localStorage` (quelques milliers de pages au plus). |
| E10 | **Copie scellée des carnets chiffrés** | FAIT | La sauvegarde du coffre (déjà chiffrée) peut être envoyée au serveur : rien de lisible n'en sort (un test examine chaque requête et le fichier stocké) ; restauration sur un navigateur sans coffre. | Manuelle et sans fusion. La synchronisation chiffrée page par page, avec hiérarchie de clés, existe depuis le point E16d (sans clé de secours : décision du 16 octobre). |

| N° | Point | État | Où on en est | Reste à faire |
|---|---|---|---|---|
| E11 | **Connecteur Nextcloud / ownCloud / WebDAV** | **FAIT** | Le serveur appelle le service pour vous (mot de passe d'application chiffré au repos, adresses privées refusées, aucune redirection suivie) ; seuls les fichiers changés sont écrits ; essayé contre un faux serveur WebDAV. | **Jamais essayé avec un vrai Nextcloud**. Écriture non atomique sur WebDAV. |
| E12 | **Connecteur GitHub** | **FAIT** | Un envoi = un commit (API Git Data), branche déplacée sans force : deux appareils ne s'écrasent pas ; essayé contre un faux GitHub. | **Jamais essayé avec le vrai GitHub** (limites de débit, branches protégées, commits signés exigés). |
| E13 | **Connecteur GitLab** | **FAIT** | Un envoi = un commit avec `last_commit_id`, listes paginées ; essayé contre un faux GitLab. | **Jamais essayé avec un vrai GitLab** ; la détection d'un conflit repose sur les mots de GitLab. |
| E14 | **Connecteur Dropbox** | **FAIT** | Autorisation dans la fenêtre de Dropbox (le serveur ne voit jamais le mot de passe), dossier d'application, jeton de rafraîchissement chiffré, révoqué quand on déconnecte ; envois par révision (un fichier modifié ailleurs n'est pas écrasé) ; essayé contre un faux Dropbox. | À vous : créer l'application Dropbox (`docs/zim-server.md`) ; **jamais essayé avec le vrai Dropbox**. Pas de validation en une fois sur plusieurs fichiers (limite de Dropbox). |
| E15 | **Connecteur Google Drive** | **FAIT** | Autorisation dans la fenêtre de Google, portée la plus étroite (`drive.file`), dossier « Zim Notebooks », vrais dossiers Drive ; essayé contre un faux Drive. | À vous : créer l'application Google ; attention, en mode « Test » Google arrête le jeton après 7 jours (à vérifier chez Google pour la publication) ; **jamais essayé avec le vrai Drive**. |
| E16a | **Synchro par page des carnets en clair** | **FAIT** | Étape 1, livrée le 16 octobre : pour chaque page, on compare ce que chaque côté a changé depuis le dernier accord (comparaison à trois : base, ici, là-bas) ; envoi et récupération des seules pages changées, suppressions comprises ; pages différentes changées sur deux appareils : fusion sans question ; même page changée des deux côtés : demandée (garder les deux avec une « copie de conflit », garder la mienne, prendre l'autre), sauf si les deux versions sont identiques. Marche pour le compte et pour tous les connecteurs ; essayée à deux navigateurs contre le vrai serveur et contre des imitations de Nextcloud, GitHub, GitLab, Dropbox et Drive. | Fusion à l'intérieur d'une page (par lignes) ; base conservée dans IndexedDB plutôt que dans `localStorage` pour de très gros carnets ; essai avec un réseau qui coupe en plein envoi et avec deux onglets sur un même carnet ; **jamais essayée sur un vrai service**. |
| E16b | **Couche de chiffrement du cœur Rust** | **FAIT** | Livrée le 16 octobre : une clé maîtresse par compte (dans le trousseau du coffre, jamais donnée en clair), une clé par carnet qui s'en déduit, des objets scellés (un par page et par pièce jointe) sous des noms qui ne disent rien, un index scellé qui porte le titre et la liste des pages ; la clé maîtresse enveloppée par la phrase du coffre (une seule phrase, décidée le 16 octobre, sans clé de secours). Huit commandes du cœur, dans la liste des commandes locales (`seal:master:make` remplace `create` à l'étape E16d : elle vérifie que la phrase est celle du coffre et ne garde rien tant que le serveur n'a pas accepté la clé). Les réponses connues sont vérifiées en natif et en wasm, reproduites octet pour octet par une autre implémentation (Python), et une implémentation Node, avec la seule phrase et la clé enveloppée, rouvre ce que le navigateur a scellé. | Utilisée par E16d. Écart avec l'étude : ChaCha20-Poly1305 (celui que le cœur a sur les deux cibles) au lieu de XChaCha20, avec un nonce aléatoire de 96 bits (limite : moins de $2^{32}$ objets par clé). Pas de renouvellement de la clé maîtresse (la phrase changée l'enveloppe de nouveau). |

| N° | Point | État | Où on en est | Reste à faire |
|---|---|---|---|---|
| E16c | **Pastilles et création en deux choix** | FAIT | Livrée le 16 octobre : une liste unique de carnets, chaque ligne avec deux pastilles à mot (« Encrypted » ; « This device only », « Folder of this computer » ou « Synced · où · état ») ; l'état (à jour, en attente d'envoi, envoi en cours, conflit « needs you », problème, hors ligne) aussi en pastille à côté du titre, réduite à un point sur un petit écran ; la fenêtre « New notebook » demande le chiffrement et l'emplacement séparément, et envoie le carnet une première fois s'il est créé sur le serveur ou un service. Un serveur injoignable ne fait pas passer un carnet synchronisé pour un carnet local. | Regrouper la liste par emplacement au-delà d'une dizaine de carnets ; pastilles et états dans les coques native et mobile (non essayées). (Les carnets chiffrés peuvent désormais être créés sur le serveur : E16d.) Depuis E16d aussi, le titre suit une connexion ou une déconnexion. |
| E16d | **Carnets chiffrés vers le serveur et les connecteurs** | FAIT | Livrée le 16 octobre : un carnet chiffré se garde sur le serveur du compte ou dans un dossier d'un service connecté, *scellé page par page* (un objet par page et pièce jointe sous un nom haché, un index scellé qui porte le titre et la liste des pages ; le serveur ne voit que « Encrypted notebook », un identifiant au hasard et des objets illisibles) et se synchronise entre appareils comme un carnet en clair (comparaison à trois, fusion, copie de conflit). La clé maîtresse est fabriquée par le cœur, gardée par le compte *enveloppée par la phrase du coffre* (comparaison-échange : deux appareils qui fabriquent la première clé en même temps n'en gardent qu'une), puis reprise dans le coffre ; un nouvel appareil crée son coffre, tape une fois la phrase qui a scellé la copie, et le carnet revient sous son nom. Un changement de phrase enveloppe la clé de nouveau (même clé) ; fait hors connexion, un message et une ligne du menu le rattrapent. Le lien d'un carnet chiffré (où il est gardé, sommes, compteur d'index vu) vit *dans le coffre*, jamais dans le stockage du navigateur. Refusés : une vieille copie remise en place, un index changé, un objet manquant, remplacé par une ancienne version, abîmé ou posé sous un autre nom. Huit scénarios à deux ou trois navigateurs contre le vrai serveur et une imitation de WebDAV, qui examinent la base du serveur, les requêtes, les fichiers du service et le stockage des navigateurs. En regardant les images sur téléphone, un défaut plus ancien est apparu et a été corrigé : le menu des carnets sortait de l'écran (lignes du bas inaccessibles) ; il est maintenant ramené dans l'écran et défile (test à la taille d'un téléphone). Mutants : 16 (cœur), 12 (clé côté serveur : deux avaient survécu, tests renforcés), 21 (client, dont un seul rattrapé après l'ajout d'un scénario) 31 sur les tests unitaires (remote scellé et son découpage en groupes, liens dans le coffre, appels de clé ; un avait survécu, test renforcé) et 4 sur le menu des carnets, tous attrapés à la fin. | **Aucune copie scellée n'a été posée sur un vrai service** (Nextcloud, GitHub, GitLab, Dropbox, Drive : imitations seulement) ; pas d'essai sur téléphone ni Windows ; carnets de dossier du bureau et applications natives hors synchro (E17). **Phrase oubliée avec une clé qu'aucun appareil ne peut ouvrir** : pas de moyen de repartir à zéro (E16i). Pas de renouvellement de la clé maîtresse. L'index entier est renvoyé à chaque envoi (environ 170 octets par page : 170 Ko pour mille pages, sur 1 Mo de quota). |
| E16e | **Déplacer une page vers un autre carnet** | EN ATTENTE | Réponse au besoin « une page secrète parmi des pages ordinaires » : un second carnet chiffré, et un moyen d'y passer une page. | Fonction à écrire (liens entrants à signaler, pièces jointes à emporter). |

| N° | Point | État | Où on en est | Reste à faire |
|---|---|---|---|---|
| E16f | **Notion de « projet » (regroupement de carnets)** | REPORTÉ | Décision du 16 octobre : l'unité reste le carnet ; une notion de projet pourra s'ajouter si le besoin se confirme. | À reprendre plus tard : regrouper plusieurs carnets, chacun gardant son chiffrement et son emplacement. |
| E16g | **Carnet mixte clair/chiffré** | ÉCARTÉ | Un carnet est soit chiffré, soit en clair, jamais les deux : les liens, rétroliens, étiquettes, recherche et arbre relient toutes les pages, et une page chiffrée à côté de pages en clair se trahirait par ces index. | Écarté ; pourrait être repris plus tard si la pertinence est démontrée. |
| E16h | **Carnet mixte local/distant** | ÉCARTÉ | Un carnet est soit local, soit distant (synchronisé), jamais les deux de façon durable ; il ne l'est mélangé que pendant un changement (envoi initial, changement d'emplacement, conflit en cours). | Écarté comme état durable ; extension possible plus tard. |
| E16i | **Repartir à zéro quand la phrase des copies scellées est oubliée** | EN ATTENTE | Constaté en livrant E16d : si la phrase est oubliée et qu'aucun appareil n'a le coffre ouvert, un nouveau coffre se voit demander l'ancienne phrase pour prendre la clé du compte, sans issue (hors suppression du compte). | Un « repartir à zéro » : supprimer la clé du compte et ses copies scellées (comparaison-échange sur la clé vue), puis en fabriquer une nouvelle ; dire aux appareils qui gardent l'ancienne clé comment l'oublier ; les copies scellées des services connectés restent à supprimer à la main. Proposé comme prochaine petite étape. |
| E17 | **Connexion au compte depuis les applications natives** | EN ATTENTE | Rien de commencé. | Connexion par adresse locale avec PKCE (bureau), permission `INTERNET` (Android), changement de la politique de contenu des coques. |
| E18 | **Décisions ouvertes de l'étude (§10)** | EN ATTENTE | Chiffrement proposé par défaut ou non ; une seule destination par carnet ; une ou deux variantes des applications natives ; éditeur légal ; sens du quota (1 Mo) ; gratuit ou payant. (Tranchées le 16 octobre : la granularité du chiffrement, le vocabulaire, la phrase unique, pas de clé de secours.) | Les autres réponses. Je suis les valeurs par défaut proposées (en clair d'abord, une destination par carnet). |
| E19 | **Cadre légal** | EN ATTENTE | Rien : éditeur, conditions d'utilisation, politique de confidentialité, RGPD (les données sont déjà exportables et supprimables). | Texte à rédiger et à faire relire avant toute ouverture au public. |
| E20 | **Durcissement avant ouverture au public** | EN ATTENTE | Protections de base écrites et testées (limites, cookies, en-têtes, adresses privées refusées). Aucune revue de sécurité extérieure. | Revue extérieure, journalisation et alertes, sauvegardes du serveur, essais de charge, procédure de mise à jour, plan en cas de fuite. |
| **F — Méthode et outils** | | | | |
| F1 | **Intégration continue (bureau)** | FAIT | GitHub Actions construit les paquets Linux et Windows à chaque changement ; nouvelles tentatives si un téléchargement échoue. | — |
| F2 | **Ce document de suivi** | FAIT | Source `docs/suivi/suivi.tex`, PDF compilé par `scripts/suivi.sh` (tectonic installé sans root). | Le tenir à jour à chaque étape (section suivante). |

# 2 Principes d'organisation des carnets

Décidés le 16 octobre 2026, avec le vocabulaire : **carnet** (l'unité), **page** (une note ; une page peut avoir des sous-pages, ce qui donne les sections), **pièce jointe**. Ces règles sont des *choix*, pas des limites techniques : elles pourront être assouplies plus tard si l'usage le justifie.

| Axe | Règle retenue | Extension possible plus tard |
|---|---|---|
| Chiffrement | Un carnet est **soit chiffré, soit en clair** : il ne contient jamais à la fois des pages chiffrées et des pages en clair. L'unité de chiffrement est le carnet (une clé par carnet). | Carnet mixte (quelques pages chiffrées dans un carnet en clair), si on trouve comment ne rien révéler par les liens, rétroliens, étiquettes, recherche et arbre. Aujourd'hui : un second carnet chiffré et une fonction « déplacer la page » (E16e). |
| Emplacement | Un carnet est **soit local** (seulement sur cet appareil), **soit distant** (synchronisé : compte du serveur ou service connecté), une seule destination à la fois. Il ne mélange pas de pages locales et de pages distantes, **sauf temporairement** pendant un changement (envoi initial, changement d'emplacement, conflit en cours de règlement). | Carnets dont certaines pages ne quittent jamais l'appareil ; plusieurs destinations à la fois pour un carnet. |
| Stockage et synchronisation | Un carnet distant est stocké **page par page** (un objet par page et par pièce jointe) avec un index ; seules les pages changées voyagent ; deux appareils qui changent des pages différentes se fusionnent seuls. Un carnet chiffré suit le même schéma, avec des objets scellés. | Découpage plus fin à l'intérieur d'une page (fusion de lignes) ; compactage de l'index pour de très gros carnets. |
| Phrase secrète | Les carnets chiffrés (locaux ou distants) réutilisent **la phrase du coffre local** : une seule phrase, jamais redemandée pour la même session. Une phrase perdue = carnets perdus (aucune clé de secours aujourd'hui ; un « repartir à zéro » pour la clé du compte reste à écrire, E16i). | Clé de secours imprimée une fois ; phrase propre à chaque carnet. |
| Interface | Une **liste unique** de carnets ; deux pastilles avec un mot : « Chiffré » et « Synchronisé (où) » ou « Cet appareil seulement » ; une icône d'état (à jour, envoi en cours, conflit, erreur). | Regroupement par destination au-delà d'une dizaine de carnets ; regroupement de carnets en « projets » (E16f). |

# 3 Décisions prises

| Date | Décision | Conséquence |
|---|---|---|
| 5 oct. 2026 | Une interface web partagée (CodeMirror 6) pour le web, le bureau et le mobile. | Pas d'interfaces natives par plateforme (point D7). |
| 6 oct. 2026 | Carnets chiffrés par le cœur Turtl en wasm, dans un Worker, un compte local par phrase secrète. | Base de tout le chiffrement qui suit. |
| 13 oct. 2026 | Pas de Flutter pour les applications natives ; coque Capacitor autour de l'application web. | Point D6 écarté. |
| 14 oct. 2026 | Bureau : Tauri 2 ; le paquet contient le build web tel quel. | Une seule interface à maintenir (D1). |
| 15 oct. 2026 | Commencer par Google et GitHub ; code envoyé seulement à l'inscription ; pas d'OVH, on reste sur ce serveur (un VPS plus tard) ; sans compte, les notes restent locales. | Points E2, E6 écartés ; E8 reporté. |
| 15 oct. 2026 | Synchroniser d'abord en clair, une destination par carnet (valeurs proposées, non contredites). | Le chiffrement fichier par fichier : E16b à E16d, faits le 16 octobre. |
| 16 oct. 2026 | Connecteurs dans l'ordre : Nextcloud/WebDAV, GitHub, GitLab, puis Dropbox et Google Drive (OAuth). | Tous faits ; reste à les essayer pour de vrai. |
| 16 oct. 2026 | Synchronisation *par page* (un objet par page et par pièce jointe, avec un index), chiffrement *par carnet* (une clé par carnet) ; plus un carnet chiffré ou clair, jamais mixte, local ou distant, jamais mixte. | Étapes E16a à E16d ; principes ci-dessous. |
| 16 oct. 2026 | Le mot « carnet » désigne l'unité (« projet » seulement comme extension future éventuelle) ; une note est une page, et les pages peuvent avoir des sous-pages. | Point E16f reporté. |
| 16 oct. 2026 | Les carnets chiffrés réutilisent la phrase du coffre local, sans seconde phrase à taper ; pas de clé de secours (phrase perdue = carnets perdus, comme le coffre). | Évite une demande de plus ; fait en E16b ; le coût en sera expliqué à la création (E16c). |

# 4 Ce qui dépend de vous

1. **Créer les applications OAuth** (étapes dans `docs/zim-server.md`) : Google et GitHub pour la connexion ; Dropbox et Google Drive pour les connecteurs. Tant que ce n'est pas fait, aucun bouton correspondant n'apparaît.
2. **Un service d'envoi d'e-mails** (SMTP) : sans lui, les codes d'inscription restent dans un fichier du serveur.
3. **Essayer pour de vrai** un Nextcloud, un dépôt GitHub (jeton limité à un dépôt privé), un projet GitLab, puis me dire ce qui coince : tout a été essayé seulement contre des imitations.
4. **Répondre aux décisions ouvertes** (point E18) et choisir l'éditeur légal (E19).
5. **Essayer sur de vrais appareils** : un téléphone Android, un iPhone si possible, une machine Windows.
6. À terme, **un VPS et un nom de domaine** pour ouvrir le service à d'autres (E3, E20). Aucune commande `sudo` n'est nécessaire pour l'instant.

# 5 Ce qui n'a jamais été vérifié

— Aucune connexion réelle à Google, GitHub, Dropbox, Google Drive, GitLab ni à un Nextcloud : les tests utilisent des imitations écrites pour eux (qui parlent le protocole tel que je le connais ; un vrai service peut répondre autrement).

— Aucune copie scellée n'a été posée sur un vrai service (Nextcloud, GitHub, GitLab, Dropbox, Drive), ni lue depuis un vrai téléphone ou une machine Windows ; aucun carnet chiffré de plus de quelques centaines de pages n'a été synchronisé.

— Aucun envoi réel d'e-mail, aucune charge, aucun essai avec plusieurs processus serveur.

— Aucun vrai téléphone, aucune machine Windows réelle, aucun Mac.

— Aucune relecture de sécurité par quelqu'un d'autre ; les mutants de test (on casse une protection pour voir si un test le remarque) ne remplacent pas cela.

— L'écriture dans plusieurs fichiers n'est pas atomique sur WebDAV, Dropbox et Google Drive (elle l'est sur GitHub et GitLab).

# 6 Journal de progression

**28 septembre 2026**

Échafaudage du projet, spécification du format Zim, lecteur/écrivain `zim-format` avec essais d'aller-retour (`22f83cf`, `e32b662`). Cœur Turtl intégré et remis en état (`ba24934`, `45d410a`). Plan du port wasm ; chiffrement identique au natif prouvé par un pont libsodium.js (`b505976`, `0b0b74e`).

**29 septembre**

Le cœur vérifie ses types pour wasm ; la vraie libsodium.js est branchée (`22f2301`, `750a02e`).

**5 octobre**

Port wasm : SQLite réel, crypto complète, async partout, le cœur en service dans un hôte JavaScript, persistance OPFS, essais dans un vrai Worker (`9cd33d0…7a4ce6d`). Pièces jointes (`931bc76`). Essais sur de vrais carnets Zim et fuzzer (`4439d04`). Éditeur CodeMirror 6 et commandes (`b11bf0f`, `7b3d601`).

**6 octobre**

Serveur statique sûr (`8a0aa09`). Règles du carnet et liaisons wasm (`4d1a1c9`, `b23f50a`). Application du carnet : arbre, liens, recherche, import/export (`8640a67`) et 31 scénarios navigateur (`00b22ec`). Pièces jointes et images (`4f5b823…55ca54e`). Comptes locaux, écritures atomiques, coffre chiffré dans l'application (`f997f29…6e5938a`).

**7 octobre**

https avec certificat local (`f064f6d`) ; phrase secrète normalisée (`af74b76`) ; corrections de revue (`1f9264a`, `a3dbf62`) ; application installable avec code épinglé (`cf7d045`) ; certificat de confiance (`5c89aa1`).

**9 octobre**

Changement de phrase, sauvegarde et restauration du coffre (`92a2ccf`, `d5985f9`, `3b3d897`). Étiquettes, journal, complétion de liens, tableaux, sommaire, calendrier (`fc66e61`, `baa6cb2`, `25522a4`).

**12 et 13 octobre**

Écriture sur téléphone : barre de mise en forme, tactile, clavier ; scénarios aussi en WebKit ; menu des actions repliées ; corrections de ce que montrent les vrais navigateurs (`1da565f…2a5cd9d`).

**14 octobre**

Coque Android (APK de débogage, essai dans une vraie WebView) (908c50c, ecf48af, b9a7ed6). Programme de bureau Tauri pour Windows et Linux, paquets construits en CI (824bb06, afec2be).

**15 octobre**

Carnets en vrais dossiers sur le bureau (17b0a53) ; CI plus robuste (25c3597, 6c33865). Étude de faisabilité des comptes et du SaaS (b40bdcf). Serveur de comptes, connexion Google/GitHub, interface des comptes (b49afd9, f97d995). Stockage des carnets par compte et synchronisation dans l'application (9693953, 2e278ec). Copie scellée des carnets chiffrés (49063d6). Connecteurs Nextcloud/WebDAV, GitHub, GitLab (f7710c6, 3673cac, eb9883d, 74db037).

**16 octobre**

Connecteurs Dropbox et Google Drive (autorisation dans la fenêtre du service, jetons chiffrés, révocation) avec leurs essais (a0ab896 et le commit qui suit) ; boutons de dialogue qui restent visibles dans une fenêtre trop haute ; ce document de suivi. Puis cadrage du chiffrement fichier par fichier (principes et décisions ci-dessus), **couche de chiffrement du cœur** (E16b : clés, objets scellés, index, clé enveloppée par la phrase du coffre, huit commandes ; vérifiée en natif, en wasm, par deux autres implémentations ; 61ce65d et les commits qui suivent) et **synchronisation par page des carnets en clair** (E16a : appels par page sur le serveur et les connecteurs, comparaison à trois, fusion, copie de conflit ; 251e5fe et les commits qui suivent). **Pastilles et création en deux choix** (E16c : deux pastilles par carnet, état de synchronisation, fenêtre de création à deux axes ; a9ce4ac et le commit qui suit). **Carnets chiffrés vers le serveur et les connecteurs** (E16d : scellés page par page, clé du compte enveloppée par la phrase du coffre avec comparaison-échange, lien gardé dans le coffre, refus des copies remises en place ou abîmées, menu des carnets ramené dans l'écran d'un téléphone ; 3c89149 et les commits qui suivent).

# 7 Risques ouverts

— **Confiance dans le code servi par le web** (risque 6) : atténuée par le service worker, non résolue ; le paquet de bureau est la meilleure garantie.

— **Notes en clair sur le serveur** : un carnet *en clair* « gardé sur le serveur » ou sur un connecteur est lisible par l'exploitant du service (les fenêtres le disent) ; un carnet *chiffré* est scellé (E16d), mais l'exploitant garde toutes les versions envoyées et peut essayer des phrases hors ligne : une phrase faible est la seule faille.

— **Phrase secrète oubliée** : aucune récupération possible (coffre et copies scellées) ; dit avant chaque activation. Et, tant que E16i n'existe pas, un compte dont la clé ne peut plus s'ouvrir ne peut plus recevoir de copie scellée (il reste la suppression du compte).

— **Dépendance à de petites imitations pour tester les services** : un vrai service peut répondre autrement ; d'où la liste de ce qui reste à essayer.

— **Fidélité à Zim** (risque 7) : l'essentiel est couvert, pas tout.

# 8 Comment ce document est tenu

À chaque étape terminée : (1) changer l'état et le « reste à faire » des points touchés dans docs/suivi/suivi.tex, ajouter les lignes nécessaires et une entrée au journal avec les identifiants de commit ; (2) mettre à jour les chiffres de tests ci-dessus (le fichier docs/suivi/releves.tex) ; (3) lancer scripts/suivi.sh, qui recompile le PDF (docs/suivi/suivi.pdf) ; (4) commiter le source et le PDF ensemble. Les totaux de la synthèse sont comptés automatiquement d'après les lignes du tableau.